

第 12 节 实操任务 1 实现 CostGuard 成本...

学习目标: cost_guard.py 能追踪成本 + 预警提醒 + 预算超标抛出

BudgetExceededError

目标文件: tests/cost_guard.py

背景

多 Agent 系统每次管线会调用多个 LLM:

1 Analyzer(~3500 tokens) → Reviewer(~3300 tokens) → 修订重跑(+2500)

一次正常管线: ~9000 tokens ≈ ¥0.012

一次带修订的管线: ~11500 tokens ≈ ¥0.015

如果死循环 10 次: ~35000+ tokens ≈ ¥0.05+

CostGuard 提供三重保护: 成本追踪 (record()) + 预警提醒 + 预算熔断 (check()) 抛出 BudgetExceededError 。

步骤 1: 用 AI 编程工具生成 cost_guard.py

以下代码可以用 OpenCode、Claude Code、Cursor、Trae 或通义灵码等任意 AI 编程工具生成。

提示词:

```
1 请帮我编写 tests/cost_guard.py, 实现多 Agent 预算守卫:
2
3 需求:
4 1. CostRecord 数据类: 记录单次 LLM 调用 (timestamp, node_name, prompt_tokens
5 2. CostGuard 类, 三重保护机制:
6     - record(node_name, usage, model=""): 记录一次 LLM 调用的 token 用量, us
7     - check() -> dict: 检查预算状态, 返回 {"status": "ok"|"warning"|"exceede
8       - 接近预算 (>= alert_threshold) 时状态为 "warning"
9       - 超出预算时抛出 BudgetExceededError 异常
10    - get_report() -> dict: 生成成本报告 (按节点分组统计)
11    - save_report(path=None): 保存成本报告到 JSON 文件
12 3. 构造函数参数: budget_yuan=1.0, alert_threshold=0.8, input_price_per_mil
13 4. BudgetExceededError 异常类
14 5. 包含 if __name__ == "__main__" 的测试代码, 验证:
15     - 成本追踪正确 (record 后 total_prompt_tokens 和 total_cost_yuan 正确)
16     - 预算超限检测 (check() 抛出 BudgetExceededError)
17     - 预警阈值触发 (check() 返回 status="warning")
```

生成的代码（参考实现）：

```
1  """CostGuard - 多 Agent 预算守卫
2
3  三重保护：成本追踪（record）+ 预警提醒 + 预算熔断（BudgetExceededError）
4  """
5
6  import time
7  from dataclasses import dataclass, field
8  from typing import Any
9
10
11 @dataclass
12 class CostRecord:
13     """单次 LLM 调用的成本记录"""
14     timestamp: float
15     node_name: str
16     prompt_tokens: int
17     completion_tokens: int
18     cost_yuan: float
19     model: str = ""
20
21
22 class BudgetExceededError(Exception):
23     """预算超标异常 - 触发熔断"""
24     pass
25
26
27 class CostGuard:
28     """成本守卫：追踪、预警、熔断
29
30     使用方式：
31         guard = CostGuard(budget_yuan=1.0)
32         guard.record("analyze", usage)    # 记录每次调用
33         guard.check()                     # 检查是否超标
34     """
35
36     def __init__(
37         self,
38         budget_yuan: float = 1.0,
39         alert_threshold: float = 0.8,
40         input_price_per_million: float = 1.0,
41         output_price_per_million: float = 2.0,
42     ) -> None:
43         self.budget_yuan = budget_yuan
44         self.alert_threshold = alert_threshold
45         self.input_price = input_price_per_million
46         self.output_price = output_price_per_million
47
```

```

48         self.records: list[CostRecord] = []
49         self.total_prompt_tokens: int = 0
50         self.total_completion_tokens: int = 0
51         self.total_cost_yuan: float = 0.0
52         self._alert_fired: bool = False
53
54     def record(self, node_name: str, usage: dict, model: str = "") -> Co
55         """记录一次 LLM 调用的 token 用量"""
56         prompt_tokens = usage.get("prompt_tokens", 0)
57         completion_tokens = usage.get("completion_tokens", 0)
58         cost = (prompt_tokens * self.input_price
59                 + completion_tokens * self.output_price) / 1_000_000
60
61         rec = CostRecord(
62             timestamp=time.time(), node_name=node_name,
63             prompt_tokens=prompt_tokens, completion_tokens=completion_to
64             cost_yuan=cost, model=model,
65         )
66         self.records.append(rec)
67         self.total_prompt_tokens += prompt_tokens
68         self.total_completion_tokens += completion_tokens
69         self.total_cost_yuan += cost
70         return rec
71
72     def check(self) -> dict[str, Any]:
73         """检查预算状态, 超标时抛出 BudgetExceededError"""
74         ratio = self.total_cost_yuan / self.budget_yuan if self.budget_y
75
76         if self.total_cost_yuan >= self.budget_yuan:
77             raise BudgetExceededError(
78                 f"成本已超出预算! 当前: ¥{self.total_cost_yuan:.4f}, "
79                 f"预算: ¥{self.budget_yuan:.2f}"
80             )
81
82         if ratio >= self.alert_threshold and not self._alert_fired:
83             self._alert_fired = True
84             status = "warning"
85             message = f"[预警] 成本已达预算的 {ratio:.0%}!"
86         else:
87             status = "ok"
88             message = f"成本正常: ¥{self.total_cost_yuan:.4f} / ¥{self.bud
89
90         return {"status": status, "total_cost": round(self.total_cost_yu
91                 "budget": self.budget_yuan, "usage_ratio": round(ratio,
92
93     def get_report(self) -> dict:
94         """生成成本报告 (按节点分组统计)"""
95         by_node: dict[str, float] = {}

```

```

96         for r in self.records:
97             by_node[r.node_name] = by_node.get(r.node_name, 0) + r.cost_
98         return {
99             "total_cost_yuan": round(self.total_cost_yuan, 6),
100             "total_prompt_tokens": self.total_prompt_tokens,
101             "total_completion_tokens": self.total_completion_tokens,
102             "total_calls": len(self.records),
103             "budget_yuan": self.budget_yuan,
104             "cost_by_node": {k: round(v, 6) for k, v in by_node.items()}
105         }
106
107
108 # --- 测试入口 ---
109 if __name__ == "__main__":
110     print("=== 测试 1: 成本追踪 ===")
111     guard = CostGuard(budget_yuan=1.0)
112     guard.record("collect", {"prompt_tokens": 100, "completion_tokens":
113     guard.record("analyze", {"prompt_tokens": 2000, "completion_tokens":
114     guard.record("review", {"prompt_tokens": 2500, "completion_tokens":
115     report = guard.get_report()
116     print(f" 调用次数: {report['total_calls']}")
117     print(f" 总成本: ¥{report['total_cost_yuan']}")
118     print(f" 按节点: {report['cost_by_node']}")
119     result = guard.check()
120     print(f" 预算状态: {result['status']}\n")
121
122     print("=== 测试 2: 预算超限 ===")
123     guard2 = CostGuard(budget_yuan=0.001)
124     guard2.record("analyze", {"prompt_tokens": 100000, "completion_token
125     try:
126         guard2.check()
127         assert False, "应该抛出 BudgetExceededError !"
128     except BudgetExceededError as e:
129         print(f" 预算超限检测通过: {e}\n")
130
131     print("=== 测试 3: 预警阈值 ===")
132     guard3 = CostGuard(budget_yuan=0.01, alert_threshold=0.5)
133     guard3.record("analyze", {"prompt_tokens": 5000, "completion_tokens"
134     result3 = guard3.check()
135     print(f" 预警状态: {result3['status']} - {result3['message']}\n")
136
137     print("所有测试通过!")

```

步骤 2: 理解代码

如果你对这段代码有疑问，可以让 AI 编程工具解释：

[请解释 cost_guard.py 的设计：](#)

- 1. 为什么成本计算公式是 `tokens * price / 1_000_000`?
- 2. `record()` 和 `check()` 为什么分开?
- 3. 为什么 `check()` 超标时抛异常而不是返回 `False`?
- 4. 在 `LangGraph` 工作流中怎么集成 `CostGuard`?

关键设计解读：

设计点	为什么这样做
三重保护	<code>record()</code> 是仪表盘，预警是黄灯， <code>BudgetExceededError</code> 是保险丝
<code>CostRecord</code> 数据类	每次调用记录完整信息（节点名、时间戳、token 数），方便按节点统计
<code>check()</code> 抛异常	超预算严重事件，异常比返回值更难被忽略
<code>get_report()</code> 按节点分组	一眼看出哪个节点最费钱，指导优化方向

步骤 3：运行测试

```
1 cd ~/ai-knowledge-base/v3-multi-agent
2 python3 tests/cost_guard.py
```

期望输出：

```
1  === 测试 1：成本追踪 ===
2      调用次数：3
3      总成本：¥0.0083
4      按节点：{'collect': 0.0002, 'analyze': 0.004, 'review': 0.0041}
5      预算状态：ok
6
7  === 测试 2：预算超限 ===
8      预算超限检测通过：成本已超出预算！当前：¥0.3000，预算：¥0.00
9
10  === 测试 3：预警阈值 ===
11      预警状态：warning - [预警] 成本已达预算的 90%！
12
13  所有测试通过！
```

步骤 4：提交到 Git

```
1 cd ~/ai-knowledge-base/v3-multi-agent
2 git add tests/cost_guard.py
```

```
3 git commit -m "feat: add CostGuard with budget control and circuit break
```

完成! CostGuard 为多 Agent 系统提供三重保护。 `record()` 追踪每次调用成本, `check()` 在接近预算时预警、超标时抛出 `BudgetExceededError` 强制停止, `get_report()` 按节点统计方便优化。